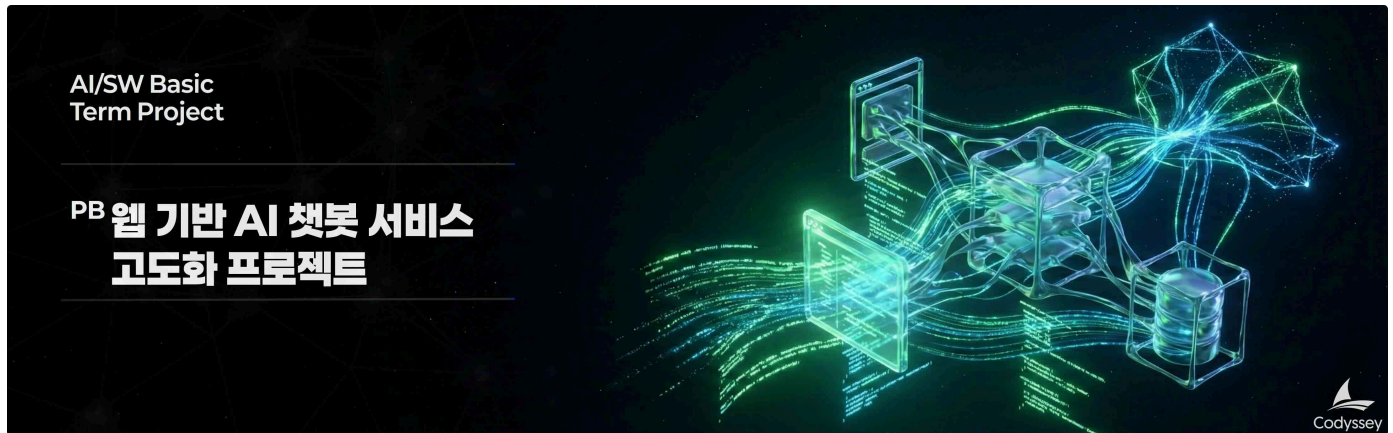


웹 기반 AI 챗봇 서비스 고도화 프로젝트

문제기술

기술적 설명



1. 미션 소개

실제 서비스는 기능이 동작하는 것만으로 완성되지 않습니다. 사용자 인증, 데이터 소유권, 외부 접근 가능한 배포, 그리고 다른 개발자가 이해할 수 있는 기술 문서까지 갖춰야 비로소 "서비스"라 부를 수 있습니다.

이 미션에서는 Project A에서 만든 AI 챗봇 MVP를 풀스택 웹 애플리케이션으로 고도화합니다. 회원가입과 로그인을 추가하여 사용자별 대화를 분리하고, 게시판 기능을 통해 AI 활용 결과를 공유할 수 있도록 확장합니다. 또한, 클라우드에 배포하여 외부에서 접속 가능한 URL로 서비스를 제공합니다.

단순 기능 추가를 넘어, ERD, API 명세서, 시스템 아키텍처 다이어그램 등 기술 문서를 작성하여 프로젝트의 설계를 체계적으로 정리하는 경험을 합니다. 이를 통해 MVP에서 실서비스 수준으로 프로젝트를 발전시키는 전체 과정을 경험하게 됩니다.

2. 최종 결과물

다음 기능이 모두 정상 동작하는 풀스택 웹 애플리케이션 1개를 완성한다.

1. 사용자 인증

- 회원가입, 로그인, 로그아웃이 동작한다.
- 비로그인 상태에서는 핵심 기능(AI 챗봇, 게시판)에 접근할 수 없다.

2. 사용자별 AI 챗봇

- 로그인한 사용자별로 대화 내역이 분리되어 저장/조회된다.
- 다른 사용자의 대화에 접근할 수 없다.

3. 게시판 CRUD

- 게시글의 목록 조회, 상세 조회, 작성, 수정, 삭제가 동작한다.
- 본인이 작성한 게시글만 수정/삭제할 수 있다.

4. 클라우드 배포

- 외부 네트워크에서 접속 가능한 URL을 통해 서비스가 제공된다.
- 회원가입부터 AI 채팅, 게시판 이용까지 전체 흐름을 수행할 수 있다.

5. 기술 문서

- ERD, API 명세서, 시스템 아키텍처 다이어그램이 포함된 기술 문서가 프로젝트에 포함되어 있다.

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- MVP를 풀스택 웹 애플리케이션으로 확장할 때 고려해야 하는 요소(인증, 데이터 소유권, 배포)를 이해하고 설명할 수 있다.
- RESTful 원칙에 따라 프론트-백엔드 간 통신 구조를 설계하고, 협업을 위한 표준 API 명세서를 직접 작성할 수 있다.
- 비즈니스 요구사항에 맞춰 ERD를 설계하고, 엔티티 간 관계 설정의 정당성과 데이터 무결성 보장 방안을 논리적으로 설명할 수 있다.
- 시스템 아키텍처를 다이어그램으로 표현하고, 각 컴포넌트의 역할과 통신 흐름을 설명할 수 있다.
- 클라우드 환경에서 웹 서비스를 배포하고, 외부 접속이 가능하도록 구성할 수 있다.

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. 사용자 인증

- 회원가입 기능이 존재해야 한다. 비밀번호는 해시 처리하여 저장한다.
- 로그인 기능이 존재해야 한다. 로그인 성공 시 인증 토큰(JWT 등)을 발급한다.
- 로그아웃 기능이 존재해야 한다.
- 인증되지 않은 사용자는 AI 챗봇과 게시판 신규 등록, 수정, 삭제 기능에 접근할 수 없어야 한다.
- 인증 상태에 따라 화면이 달라져야 한다(로그인 전/후 UI 구분).

2. 사용자별 AI 챗봇

- Project A의 챗봇 기능을 기반으로, 인증된 사용자별로 대화가 분리되어야 한다.
- 각 사용자는 본인의 대화 세션만 조회할 수 있어야 한다.
- 대화 세션 생성, 목록 조회, 메시지 내역 조회, 메시지 전송 및 AI 응답 생성이 동작해야 한다.
- 다른 사용자의 대화 세션에 접근하면 적절한 에러 응답(403 또는 404)을 반환해야 한다.

3. 게시판 CRUD

- 게시글은 최소한 제목, 내용, 작성자, 작성일시를 포함한다.
- 다음 기능이 모두 동작해야 한다.
 - 게시글 목록 조회 (전체 사용자 조회 가능)
 - 게시글 상세 조회 (전체 사용자 조회 가능)
 - 게시글 작성 (로그인 사용자만 가능)
 - 게시글 수정 (본인 작성 게시글만 가능)
 - 게시글 삭제 (본인 작성 게시글만 가능)
- 게시글 API는 RESTful 규칙을 따른다.

4. RESTful API 설계

- 모든 백엔드 기능은 RESTful API로 구현한다.
- URL 설계, HTTP 메서드, 상태 코드가 REST 규칙에 맞게 적용되어야 한다.
- 프론트엔드는 이 API를 통해 백엔드와 통신한다.

5. 클라우드 배포

- 서비스를 클라우드 환경(AWS, GCP, Azure 또는 기타 클라우드/PaaS)에 배포한다.
- 외부 네트워크에서 접속 가능한 URL(퍼블릭 IP 또는 도메인)을 통해 서비스에 접근할 수 있어야 한다.
- 배포된 환경에서 회원가입, 로그인, AI 채팅, 게시판 기능이 모두 정상 동작해야 한다.

6. 인프라 및 협업

- GitHub 저장소에 `feature` → `develop` → `main` 으로 이어지는 브랜치 전략을 적용한 커밋 내역이 존재해야 한다.
- 기능 단위로 브랜치를 생성하고, Pull Request를 통해 병합한 이력이 확인 가능해야 한다.

7. 기술 문서

- 다음 기술 문서를 작성하여 프로젝트에 포함한다.

a. ERD (Entity-Relationship Diagram)

- 프로젝트의 모든 엔티티와 관계를 표현한다.

- 각 엔티티의 주요 필드와 PK/FK를 명시한다.

b. API 명세서

- 모든 API 엔드포인트를 문서화한다.
- 각 API에 대해 URL, HTTP 메서드, 요청 파라미터/본문, 응답 형식, 상태 코드를 명시한다.

c. 시스템 아키텍처 다이어그램

- 프론트엔드, 백엔드, 데이터베이스, AI API, 클라우드 인프라의 구성과 통신 흐름을 표현한다.

8. 제출물 구성

- 전체 프로젝트 소스코드 (GitHub 저장소 링크)
- 배포된 서비스 URL
- 기술 문서 (ERD, API 명세서, 시스템 아키텍처 다이어그램)
- README.md에 다음 내용을 포함한다.
 - 서비스 소개
 - 기술 스택
 - 배포 URL
 - 로컬 실행 방법
 - 환경변수 설정 방법
 - 프로젝트 구조 설명

개발환경

5. 개발 환경

- Python 3.10 이상

제약조건

6. 제약 사항

- 보안 및 비밀 관리

- API 키, 데이터베이스 비밀번호 등 민감 정보는 소스 코드에 직접 작성하지 않는다.
- 모든 민감 정보는 `.env` 파일을 통해 환경 변수로 관리해야 한다.

- 협업 규칙

- 모든 팀원은 최소 10회 이상의 유의미한 커밋을 생성해야 한다.
- 기술 문서(README 또는 별도 문서)에 팀 구성원 역할과 개인별 작업 요약을 반드시 포함해야 한다.
- `main` 브랜치에는 직접 코드를 푸시할 수 없으며, 반드시 Pull Request를 통해서만 병합해야 한다.

- 배포

- 비용 관리에 주의하며, 실습 종료 후 불필요한 리소스를 정리한다.

Test Case

7. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 문구와 디자인은 달라도 된다.

- 서비스 흐름 예시

- CSS
1. 회원가입 → 이메일, 비밀번호, 닉네임 입력
 2. 로그인 → 토큰 발급 → 인증 상태로 전환
 3. AI 챗봇 → 새 대화 시작 → 메시지 주고받기 → 대화 내역 저장
 4. 게시판 → 게시글 작성 → 목록에서 확인 → 수정/삭제
 5. 다른 기기/브라우저에서 배포 URL로 접속 → 동일 서비스 이용 가능

- ERD 예시

CSS

```
[User] 1 ——— N [ChatSession] 1 ——— N [Message]
|
└── 1 ——— N [Post]
```

User: id(PK), email, password_hash, username, created_at
ChatSession: id(PK), user_id(FK), title, created_at
Message: id(PK), session_id(FK), role, content, created_at

Post: id(PK), user_id(FK), title, content, created_at, updated_at

- API 명세서 예시

메서드	URL	설명	인증
POST	/auth/signup	회원가입	X
POST	/auth/login	로그인	X
GET	/chat/sessions	대화 세션 목록	O
POST	/chat/sessions	새 대화 세션 생성	O
POST	/chat/sessions/{id}/messages	메시지 전송	O
GET	/chat/sessions/{id}/messages	메시지 내역 조회	O
GET	/posts	게시글 목록	O
POST	/posts	게시글 작성	O
GET	/posts/{id}	게시글 상세	O
PUT	/posts/{id}	게시글 수정	O (본인만)
DELETE	/posts/{id}	게시글 삭제	O (본인만)

- 시스템 아키텍처 예시

